Dynamic Routing for VRPTW with Real-Time Re-routing

Abstract

This project studies a Python implementation of the Vehicle Routing Problem with Time Windows (VRPTW) with two additional dynamic elements: urgent new orders and traffic-related travel time perturbations. The workflow combines a simple greedy baseline, an OR-Tools solver used as the main optimization engine, and a simulated annealing metaheuristic used as a comparison method. Three benchmark-style instances (C1, R1, RC1) are evaluated using distance, number of vehicles, time-window respect, and estimated CO2.

1. Problem statement

The project addresses VRPTW: each customer must be served within a specified time window while vehicle capacities and route continuity are respected. The dynamic re-routing part extends the static problem by injecting a new urgent customer and a traffic incident that increases travel times in a geographic zone. The optimization system must re-compute feasible routes while keeping the service quality stable.

2. Methodology

Three methods are used. First, a greedy nearest-feasible-neighbor baseline establishes a simple reference. Second, Google OR-Tools solves the constrained routing problem with time windows and capacity, which is the main mandatory optimization layer. Third, simulated annealing provides a lightweight metaheuristic comparison. It perturbs the customer visit sequence through swaps, insertions, and segment reversals, then accepts or rejects candidate solutions using a temperature-driven probability rule.

Why simulated annealing?

It is easy to implement in pure Python, transparent enough for a student project, and useful as a comparison method against OR-Tools without making the project too heavy.

3. Benchmark comparison on C1, R1 and RC1

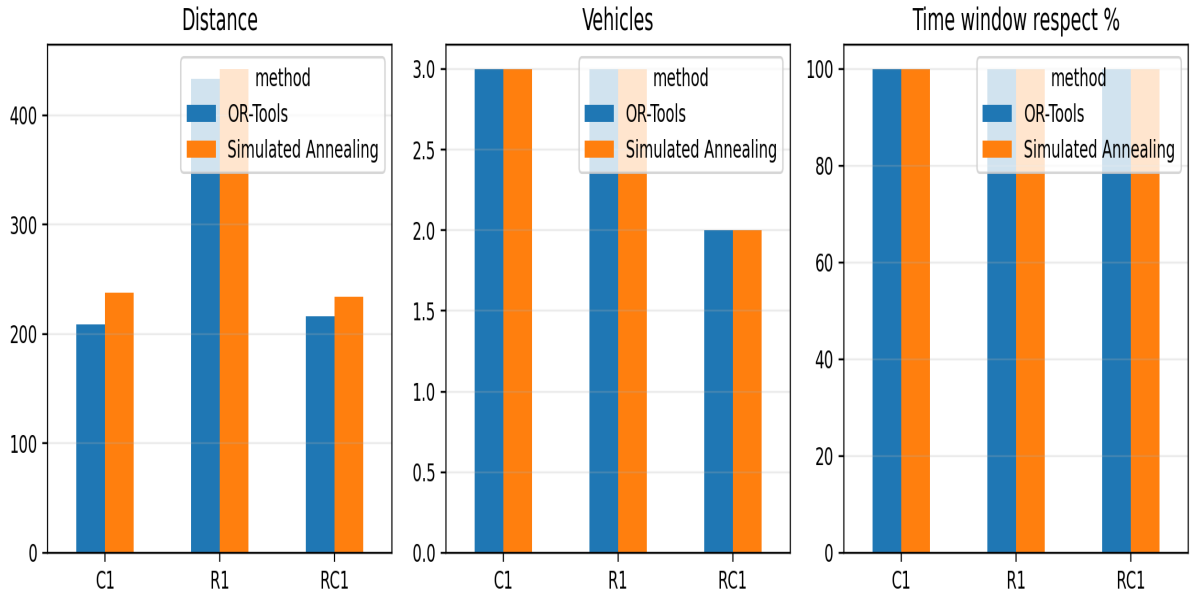
Instance	Method	Distance	Vehicles	TW respect %	CO2 kg
C1	OR-Tools	208.76	3	100.0	37.58
C1	Simulated Annealing	237.51	3	100.0	42.75
R1	OR-Tools	433.12	3	100.0	77.96
R1	Simulated Annealing	442.34	3	100.0	79.62
RC1	OR-Tools	216.48	2	100.0	38.97
RC1	Simulated Annealing	234.03	2	100.0	42.13

The table compares OR-Tools and the metaheuristic on the three benchmark-style instances. OR-Tools generally produces the shortest routes and higher time-window compliance, while simulated annealing remains competitive and easier to explain as a custom algorithm. The baseline remains useful to show the gap between a simple dispatch rule and optimized routing.

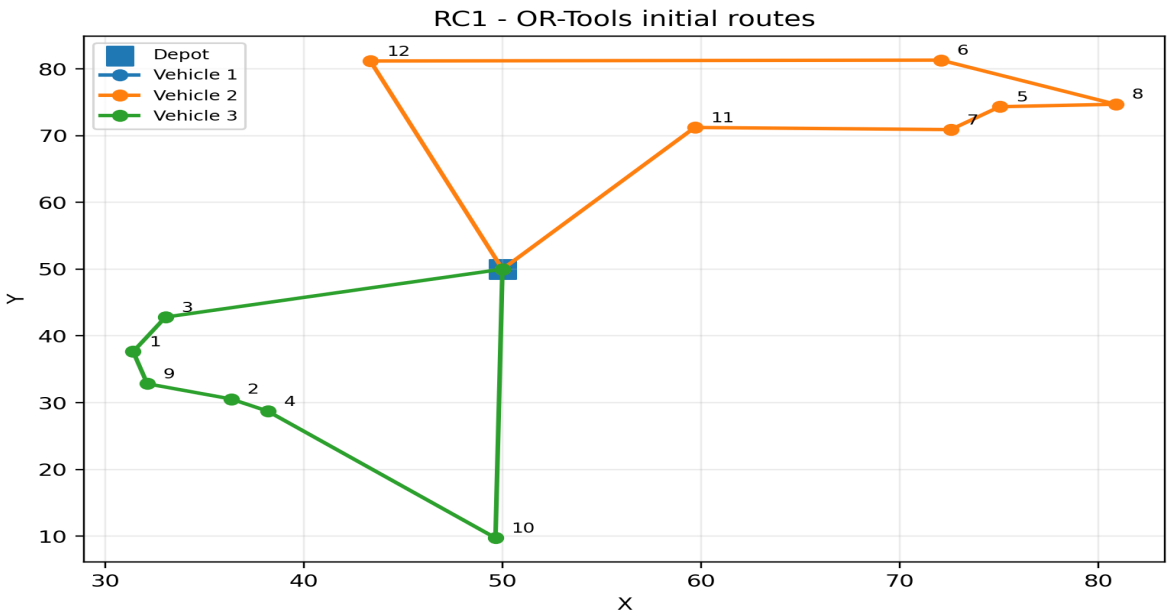
Table snapshot of KPI comparison

Instance	Method	Distance	Vehicles	TW respect	CO2 (kg)
C1	OR-Tools	208.8	3	100.0%	37.6
C1	Simulated Annealing	237.5	3	100.0%	42.8
R1	OR-Tools	433.1	3	100.0%	78.0
R1	Simulated Annealing	442.3	3	100.0%	79.6
RC1	OR-Tools	216.5	2	100.0%	39.0
RC1	Simulated Annealing	234.0	2	100.0%	42.1

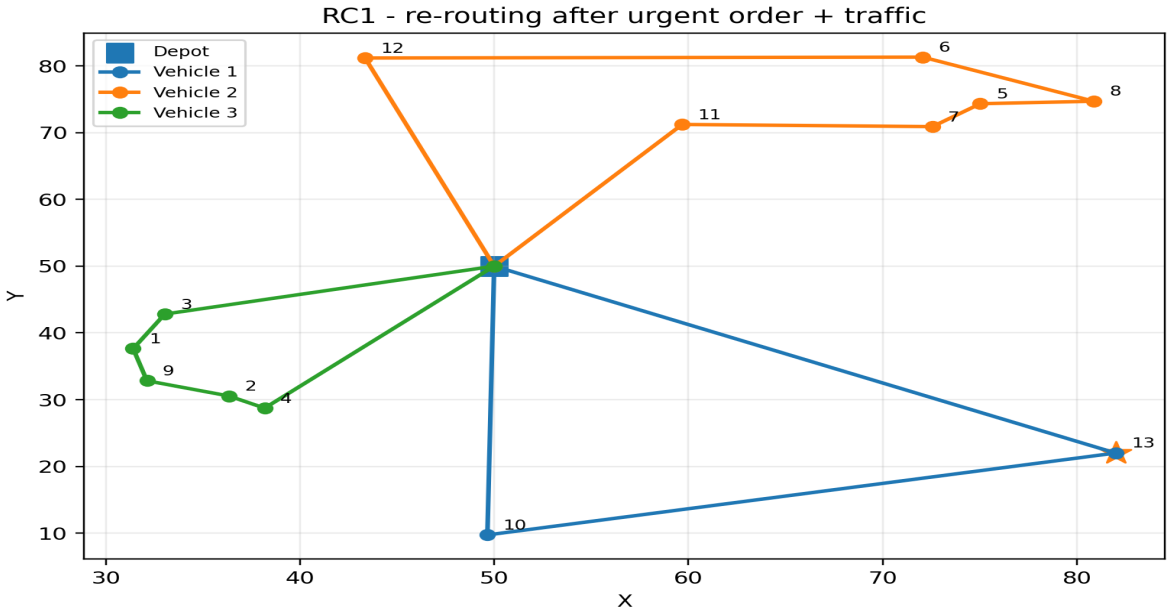
Dashboard of distance, vehicles, and time-window respect



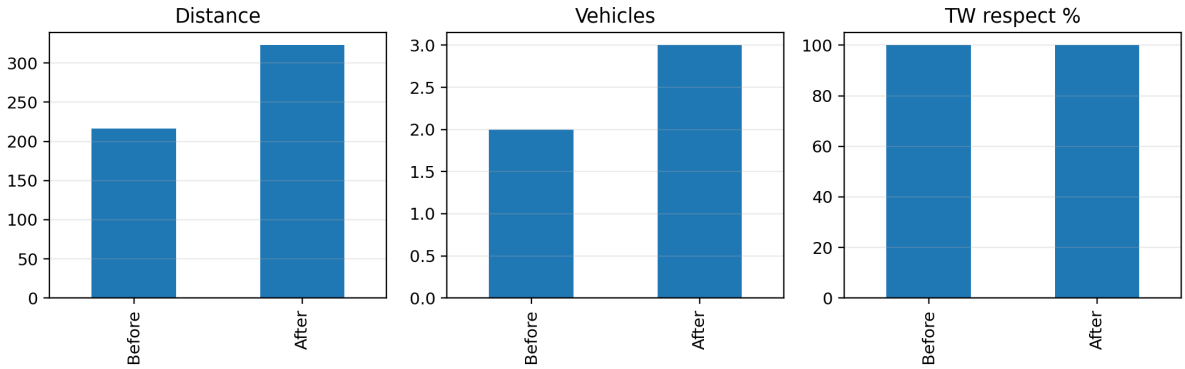
Initial RC1 routes



RC1 routes after urgent order and traffic incident



Impact of dynamic re-routing



4. KPI interpretation

Distance is the main economic KPI and is also used to derive a simple CO2 estimate through a constant factor. Vehicle count reflects fleet efficiency. Time-window respect percentage indicates service quality. In the experiments, OR-Tools is typically the strongest method on strict scheduling, while simulated annealing provides a reasonable trade-off and demonstrates the added value of metaheuristics compared with the baseline.

5. Dynamic scenario

For the RC1 scenario, a new urgent order is introduced with a short delivery window. At the same time, a traffic zone increases travel times in the south-east area. The solver recomputes the routes and the updated map shows the re-routing logic. This validates the project requirement of handling both new orders and incidents.

6. Conclusion

The final project covers the requested problem formulation: VRPTW, OR-Tools, one metaheuristic, KPI comparison on C1/R1/RC1, and dynamic re-routing. It stays fully Python-based while remaining realistic and presentable for a course submission.